Społeczna Akademia Nauk

Instytut Technologii Informatycznych

Kierunek studiów:

INFORMATYKA

Przedmiot:

Systemy Szkieletowe

System do zarządzania sylabusami uczelni wyższej

Kacper Juśkiewicz
Nr albumu: 114725
Grupa: 4

Prowadzący:
mgr inż. Krystian Gumiński

Łódź 2026

Spis treści

1 Wstęp **2**

2 Cel projektu **3**
2.1 Problem . . . 3
2.2 Cele . . . 3
2.3 Wymagania funkcjonalne . . . 3
2.4 Wymagania niefunkcjonalne . . . 4

3 Przegląd istniejących rozwiązań **6**
3.1 USOS – Uniwersytecki System Obsługi Studiów . . . 6
3.2 Akademus . . . 6
3.3 Uczelnia.XP . . . 6
3.4 Podsumowanie przeglądu . . . 7

4 Opis architektury **8**
4.1 Architektura ogólna . . . 8
4.2 Warstwy backendu . . . 8
4.3 Komunikacja między komponentami . . . 9
4.4 Zastosowane technologie . . . 11

5 Opis implementacji **12**
5.1 Frontend . . . 12
5.2 Backend . . . 16
5.2.1 Uwierzytelnianie i autoryzacja . . . 16
5.2.2 Research: logowanie kontem uczelnianym . . . 17
5.2.3 Wersjonowanie i historia zmian . . . 17
5.2.4 Walidacja danych . . . 18
5.2.5 Wyszukiwanie i siatka przedmiotów . . . 18
5.2.6 Eksport do PDF i DOCX . . . 19
5.3 Baza danych . . . 19
5.4 Docker . . . 20
5.5 Testy . . . 21
5.6 Monitoring . . . 22

6 Podsumowanie i wnioski **23**
6.1 Napotkane wyzwania . . . 23
6.2 Możliwości dalszego rozwoju . . . 23

Bibliografia **28**

1 Wstęp

Sylabus jest jednym z podstawowych dokumentów opisujących proces kształcenia na uczelni wyższej. Zawiera informacje o przedmiocie: jego cele, efekty uczenia się, wymagania wstępne, formy zajęć wraz z liczbą godzin, punkty ECTS, sposób oceniania oraz wykaz literatury. Obowiązek udostępniania programów studiów, a w ich ramach opisów przedmiotów, wynika z ustawy Prawo o szkolnictwie wyższym i nauce [1], natomiast sposób przeliczania nakładu pracy studenta na punkty ECTS opisuje europejski system ECTS [2].

W praktyce wiele uczelni nadal przygotowuje sylabusy w edytorach tekstu i przesyła je pocztą elektroniczną. Prowadzi to do typowych problemów: istnieje wiele kopii tego samego dokumentu, trudno ustalić, która wersja jest aktualna, a zmiany wprowadzone przez prowadzącego nie zostawiają śladu. Dodatkowo uczelnie posiadające kilka ośrodków (np. w różnych miastach) muszą utrzymywać osobne sylabusy tego samego przedmiotu, które z czasem zaczynają się różnić. Sprawdzenie, czy liczba godzin zajęć zgadza się z punktami ECTS i z siatką studiów, wymaga ręcznego przeliczania.

Niniejsza praca opisuje projekt i implementację aplikacji **SyllabusApp** – webowego systemu do zarządzania sylabusami uczelni wyższej. System umożliwia tworzenie i edycję sylabusów w formularzach, automatyczne wersjonowanie z historią zmian, wyszukiwanie i filtrowanie, prezentację siatki przedmiotów kierunku, walidację liczby godzin oraz eksport sylabusa do formatów PDF i DOCX.

Aplikacja została zbudowana na platformie .NET 10: backend w technologii ASP.NET Core Web API [3], frontend w Blazor WebAssembly [4], a dane przechowywane są w bazie PostgreSQL [5] obsługiwanej przez Entity Framework Core [6]. Całość jest uruchamiana w kontenerach Docker [7], posiada testy jednostkowe oraz mechanizm monitorowania stanu (health checks).

Projekt został zrealizowany indywidualnie – autor odpowiada za całość prac (100% wkładu): analizę wymagań, projekt architektury, implementację backendu, frontendu i bazy danych, konteneryzację, testy oraz dokumentację.

2 Cel projektu

2.1 Problem

Głównym problemem, który rozwiązuje aplikacja, jest brak jednego, wiarygodnego źródła informacji o sylabusach na uczelni. Dokumenty rozproszone w plikach i skrzynkach pocztowych utrudniają pracę zarówno prowadzącym (którzy co roku aktualizują te same treści), jak i władzom wydziału (które muszą kontrolować zgodność sylabusów z programem studiów), a także studentom (którzy szukają aktualnych informacji o przedmiocie).

2.2 Cele

Celem projektu było zbudowanie aplikacji webowej, która:

- przechowuje sylabusy w ustrukturyzowanej postaci w bazie danych, powiązane z ośrodkiem, wydziałem, kierunkiem, przedmiotem i rokiem akademickim,
- zapewnia formularze do wprowadzania wszystkich elementów sylabusa,
- automatycznie zapisuje każdą zmianę jako nową wersję, tak aby zawsze można było odtworzyć historię dokumentu,
- kontroluje poprawność danych, w szczególności zgodność liczby godzin z punktami ECTS,
- pozwala szybko odnaleźć sylabus według wielu kryteriów,
- prezentuje siatkę przedmiotów kierunku w podziale na semestry,
- umożliwia eksport sylabusa do formatów PDF i DOCX,
- zapewnia bezpieczny dostęp z podziałem na role użytkowników.

2.3 Wymagania funkcjonalne

Tabela 1 zestawia wymagania funkcjonalne postawione przed projektem wraz ze sposobem ich realizacji w aplikacji.

Tabela 1: Wymagania funkcjonalne i sposób ich realizacji

Wymaganie	**Realizacja w SyllabusApp**
Logowanie przy użyciu konta uczelnianego	Przeprowadzono research rozwiązań (OpenID Connect z Microsoft Entra ID, USOS API) – opisany w podrozdziale 5.2.2; architektura JWT pozwala dodać zewnętrznego dostawcę tożsamości bez zmian w pozostałej części systemu
Rejestracja i logowanie loginem i hasłem	ASP.NET Core Identity (hasła haszowane), token JWT, endpointy `/auth/register`, `/auth/login`, `/auth/me`
Formularze sylabusa (opis, cele, efekty, wymagania, literatura, ocenianie, godziny)	Formularz sylabusa oraz sekcje efektów uczenia się, literatury i nakładu pracy na stronie szczegółów
Wersjonowanie sylabusów	Każde utworzenie i edycja zapisuje nową wersję (tabela `SyllabusVersions`) z numerem, autorem, datą, opisem zmian i migawką JSON
Historia zmian	Tabela „Historia zmian" w szczegółach sylabusa, endpoint `GET /syllabuses/{id}/versions`
Różne sylabusy dla ośrodków i roku wprowadzenia	Hierarchia Ośrodek → Wydział → Kierunek → Przedmiot → Sylabus z polem rok akademicki
Siatka przedmiotów	Widok „Siatka przedmiotów" – semestry, godziny wykładów/ćwiczeń/laboratoriów, ECTS, sumy
Walidacja danych (liczba godzin)	Reguły `SyllabusRules`: zakres ECTS, nieujemne godziny, godziny zajęć $\leq$ ECTS $\times$ 30, kontrola nakładu pracy 25–30 h/ECTS
Wyszukiwanie i filtrowanie	Wyszukiwanie pełnotekstowe oraz filtry: ośrodek, kierunek, rok akademicki, status, semestr
Eksport do PDF i DOCX	Biblioteki QuestPDF i Open XML SDK, endpointy `/export/pdf` i `/export/docx`

2.4 Wymagania niefunkcjonalne

- **Bezpieczeństwo** – dostęp do API wyłącznie z ważnym tokenem JWT, autoryzacja oparta na rolach (Administrator, Dziekan, Wykładowca, Student), hasła przechowywane w postaci skrótów.
- **Przenośność i łatwość wdrożenia** – cały system (baza, API, frontend) uruchamiany jednym poleceniem `docker compose up`.
- **Utrzymywalność** – podział na warstwy zgodnie z zasadami czystej architektury [8], jednolity wzorzec dla wszystkich encji (DTO, serwis, kontroler, klient API, strony).
- **Niezawodność i obserwowalność** – endpoint `/health` sprawdzający połączenie z bazą, healthchecki kontenerów.

- **Jakość** – testy jednostkowe logiki biznesowej uruchamiane poleceniem `dotnet test`.
- **Użyteczność** – responsywny interfejs w języku polskim, czytelne komunikaty błędów walidacji.

3 Przegląd istniejących rozwiązań

Na polskim rynku funkcjonuje kilka systemów obsługujących sylabusy, zwykle jako moduły większych systemów zarządzania uczelnią. W rozdziale przeanalizowano trzy z nich.

3.1 USOS – Uniwersytecki System Obsługi Studiów

USOS [9] jest rozwijany przez Międzyuniwersyteckie Centrum Informatyzacji i wykorzystywany przez kilkadziesiąt polskich uczelni. Opisy przedmiotów (sylabusy) są przechowywane w katalogu przedmiotów i publikowane w serwisie USOSweb. System obsługuje cykle dydaktyczne, efekty uczenia się powiązane z Polską Ramą Kwalifikacji oraz rozbudowane programy studiów. Udostępnia także publiczne API (USOS API) [10] z autoryzacją OAuth, co umożliwia integrację z aplikacjami zewnętrznymi.

Zalety: bardzo szeroki zakres funkcji, integracja z rekrutacją, ocenami i planem zajęć, ugruntowana pozycja na rynku. **Wady:** duża złożoność wdrożenia i utrzymania, interfejs edycji sylabusów uznawany przez użytkowników za mało intuicyjny, rozwiązanie przeznaczone głównie dla uczelni publicznych zrzeszonych w konsorcjum.

3.2 Akademus

Akademus [11] to komercyjny system wspomagający zarządzanie uczelnią, oferujący m.in. moduł programów studiów i sylabusów. Pozwala tworzyć karty przedmiotów, przypisywać efekty uczenia się, generować dokumenty i publikować je na stronie uczelni.

Zalety: gotowe szablony dokumentów, generowanie wydruków, moduł sprawozdawczy. **Wady:** rozwiązanie zamknięte i płatne, ograniczone możliwości dostosowania, sylabus jest częścią dużego, monolitycznego systemu, co utrudnia wdrożenie samego modułu.

3.3 Uczelnia.XP

Uczelnia.XP [12] firmy Asseco Data Systems to zintegrowany system obsługi toku studiów. Zawiera moduł planów i programów studiów, w ramach którego przygotowywane są sylabusy zgodne z PRK, wraz z obiegiem zatwierdzania dokumentów.

Zalety: pełna integracja z dziekanatem, obieg dokumentów i akceptacji, wsparcie producenta. **Wady:** wysoki koszt licencji, ciężki interfejs, zależność od jednego dostawcy.

3.4 Podsumowanie przeglądu

Tabela 2: Porównanie istniejących rozwiązań z SyllabusApp

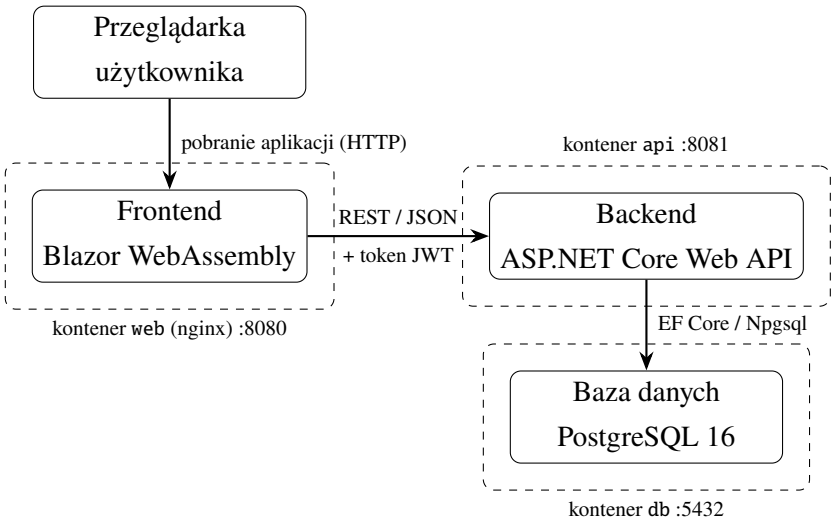
Cecha	**USOS**	**Akademus**	**Uczelnia.XP**	**SyllabusApp**
Formularze sylabusa	tak	tak	tak	tak
Historia wersji sylabusa	częściowo	częściowo	tak	tak
Siatka przedmiotów	tak	tak	tak	tak
Walidacja godzin/ECTS	tak	tak	tak	tak
Eksport PDF	tak	tak	tak	tak
Eksport DOCX	nie	tak	tak	tak
Wiele ośrodków	tak	tak	tak	tak
Lekkie wdrożenie (Docker)	nie	nie	nie	tak
Otwarte REST API	tak	nie	nie	tak
Koszt licencji	konsorcjum	płatny	płatny	brak

Istniejące rozwiązania są rozbudowanymi systemami klasy ERP dla uczelni, w których sylabus stanowi jedynie mały fragment. SyllabusApp koncentruje się wyłącznie na sylabusach, dzięki czemu jest prosty we wdrożeniu (jeden plik `docker-compose.yml`), posiada nowoczesny interfejs i otwarte REST API, a jednocześnie realizuje kluczowe funkcje: wersjonowanie, siatkę przedmiotów, walidację i eksport dokumentów.

4 Opis architektury

4.1 Architektura ogólna

System ma architekturę klient–serwer z trzema głównymi komponentami: aplikacją frontendową uruchamianą w przeglądarce, serwerem REST API oraz bazą danych. Każdy komponent działa w osobnym kontenerze Docker (rys. 1).



Rysunek 1: Architektura systemu SyllabusApp

Aplikacja Blazor WebAssembly jest zbiorem plików statycznych serwowanych przez serwer nginx. Po pobraniu do przeglądarki kod .NET wykonuje się bezpośrednio w środowisku WebAssembly i komunikuje się z API za pomocą żądań HTTP w formacie JSON. API weryfikuje token JWT, wykonuje logikę biznesową i zapisuje dane w bazie PostgreSQL.

4.2 Warstwy backendu

Kod backendu podzielono na cztery projekty zgodnie z zasadami czystej architektury [8]. Zależności są skierowane do środka – warstwa domeny nie zależy od żadnej innej warstwy (tab. 3).

Tabela 3: Projekty (warstwy) rozwiązania

Projekt	**Odpowiedzialność**	**Zależy od**
`SyllabusApp.Domain`	Encje (Sylabus, Przedmiot, Kierunek, . . .), enumeracje, stałe ról	–
`SyllabusApp.Application`	DTO, mapowania, interfejsy serwisów, reguły walidacji `SyllabusRules`	Domain
`SyllabusApp.Infrastructure`	`DbContext`, migracje, implementacje serwisów, generowanie JWT, eksport PDF/DOCX	Application
`SyllabusApp.Api`	Kontrolery REST, konfiguracja uwierzytelniania, CORS, OpenAPI, health checks	Application, Infrastructure
`SyllabusApp.Web`	Frontend Blazor WebAssembly: strony, klienci API, obsługa logowania	Application
`SyllabusApp.Tests`	Testy jednostkowe (xUnit)	Application, Infrastructure

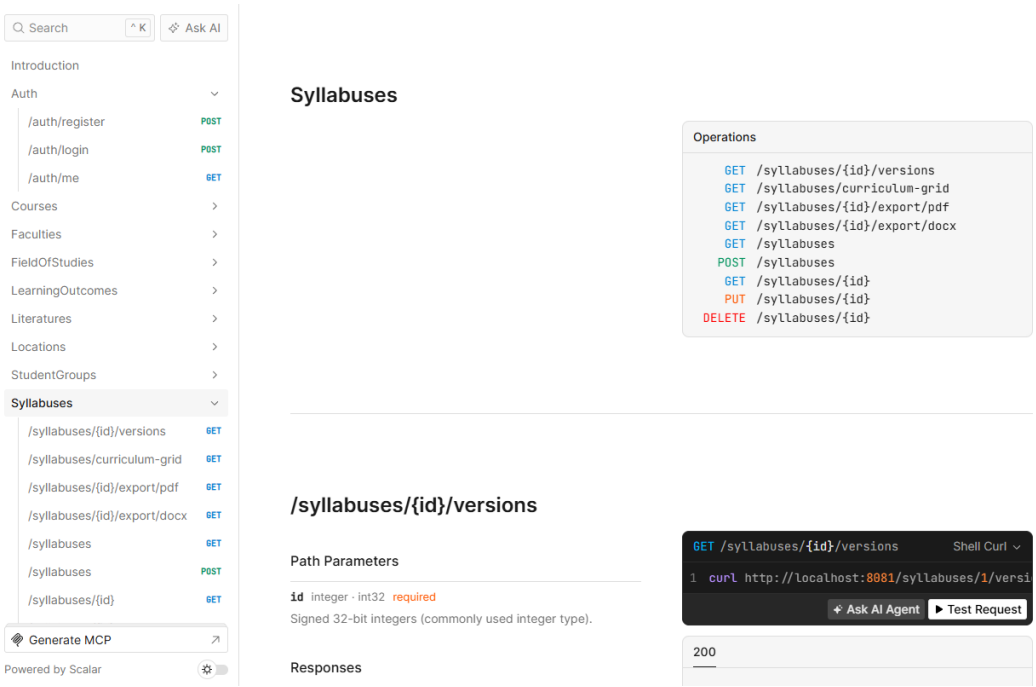
Ważną decyzją projektową jest współdzielenie projektu `Application` pomiędzy backendem a frontendem. Te same klasy DTO (wraz z atrybutami walidacji) są używane po obu stronach, dzięki czemu reguły walidacji formularza są sprawdzane już w przeglądarce, a następnie ponownie na serwerze – bez duplikowania kodu.

4.3 Komunikacja między komponentami

Frontend komunikuje się z backendem wyłącznie przez REST API. Każde żądanie (poza logowaniem i rejestracją) zawiera nagłówek `Authorization: Bearer <token>`. Token JWT [13] jest wydawany przez endpoint `/auth/login`, przechowywany w `localStorage` przeglądarki i dołączany automatycznie przez `AuthorizationMessageHandler`. Enumeracje są przesyłane jako tekst (np. `"Draft"`), co zwiększa czytelność API. Dokumentacja API jest generowana automatycznie w formacie OpenAPI i prezentowana przez interfejs Scalar [14] (rys. 2).

Tabela 4: Najważniejsze endpointy REST API

Metoda	Ścieżka	Opis
POST	`/auth/register`, `/auth/login`	Rejestracja i logowanie, zwraca token JWT
GET	`/auth/me`	Dane zalogowanego użytkownika
GET	`/syllabuses?search=&locationId=...`	Lista sylabusów z wyszukiwaniem i filtrami
GET/POST	`/syllabuses`	Odczyt i tworzenie sylabusów
GET/PUT/DELETE	`/syllabuses/{id}`	Szczegóły, edycja (nowa wersja), usunięcie
GET	`/syllabuses/{id}/versions`	Historia wersji sylabusa
GET	`/syllabuses/{id}/export/pdf`	Eksport do PDF
GET	`/syllabuses/{id}/export/docx`	Eksport do DOCX
GET	`/syllabuses/curriculum-grid`	Siatka przedmiotów kierunku
CRUD	`/locations`, `/faculties`, ...	Słowniki: ośrodki, wydziały, kierunki, przedmioty, grupy
CRUD	`/learning-outcomes`, `/literatures`, `/workload-items`	Elementy sylabusa
GET	`/health`	Stan aplikacji i połączenia z bazą



Rysunek 2: Interaktywna dokumentacja API (Scalar)

4.4 Zastosowane technologie

Tabela 5: Stos technologiczny

Obszar	**Technologia**
Język i platforma	C# 14, .NET 10
Backend	ASP.NET Core Web API, ASP.NET Core Identity, JWT Bearer
Frontend	Blazor WebAssembly, Tailwind CSS 4 [15]
Baza danych	PostgreSQL 16, Entity Framework Core 10, Npgsql [16]
Dokumentacja API	OpenAPI (Microsoft.AspNetCore.OpenApi), Scalar
Eksport dokumentów	QuestPDF [17], DocumentFormat.OpenXml [18]
Testy	xUnit [19], EF Core InMemory
Konteneryzacja	Docker, Docker Compose, nginx
Monitoring	ASP.NET Core Health Checks [20], healthchecki Docker

5 Opis implementacji

W rozdziale omówiono kluczowe aspekty implementacyjne projektu z podziałem na frontend, backend, bazę danych, konteneryzację, testy i monitoring.

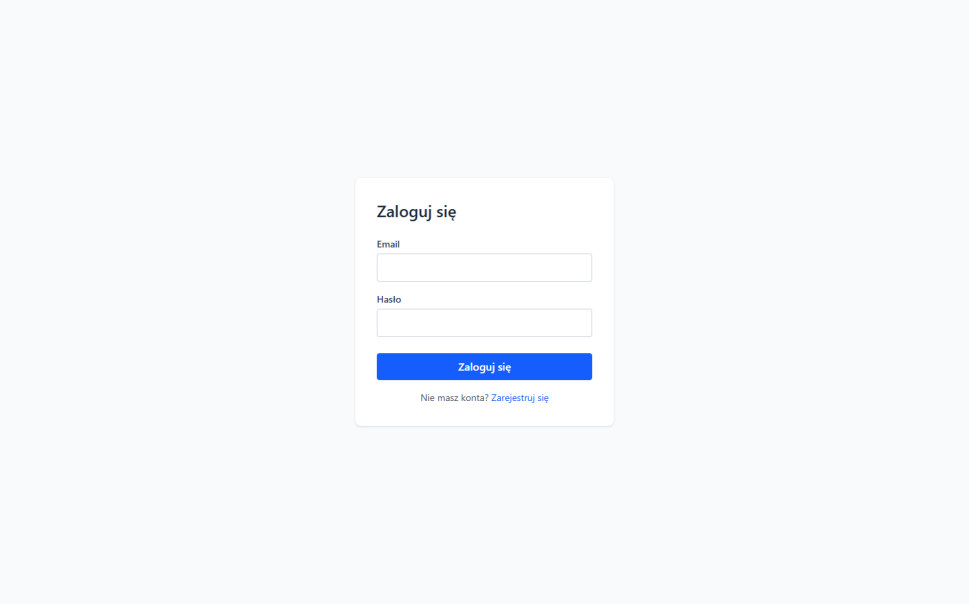
5.1 Frontend

Frontend zrealizowano jako aplikację jednostronicową (SPA) w technologii Blazor WebAssembly [4]. Pozwala to pisać logikę interfejsu w języku C# i współdzielić klasy DTO z backendem. Wygląd zbudowano w oparciu o Tailwind CSS [15] – w projekcie zdefiniowano kilka klas komponentów (`btn-primary`, `form-input`, `data-table`), które zapewniają spójny wygląd wszystkich stron.

Struktura frontendu:

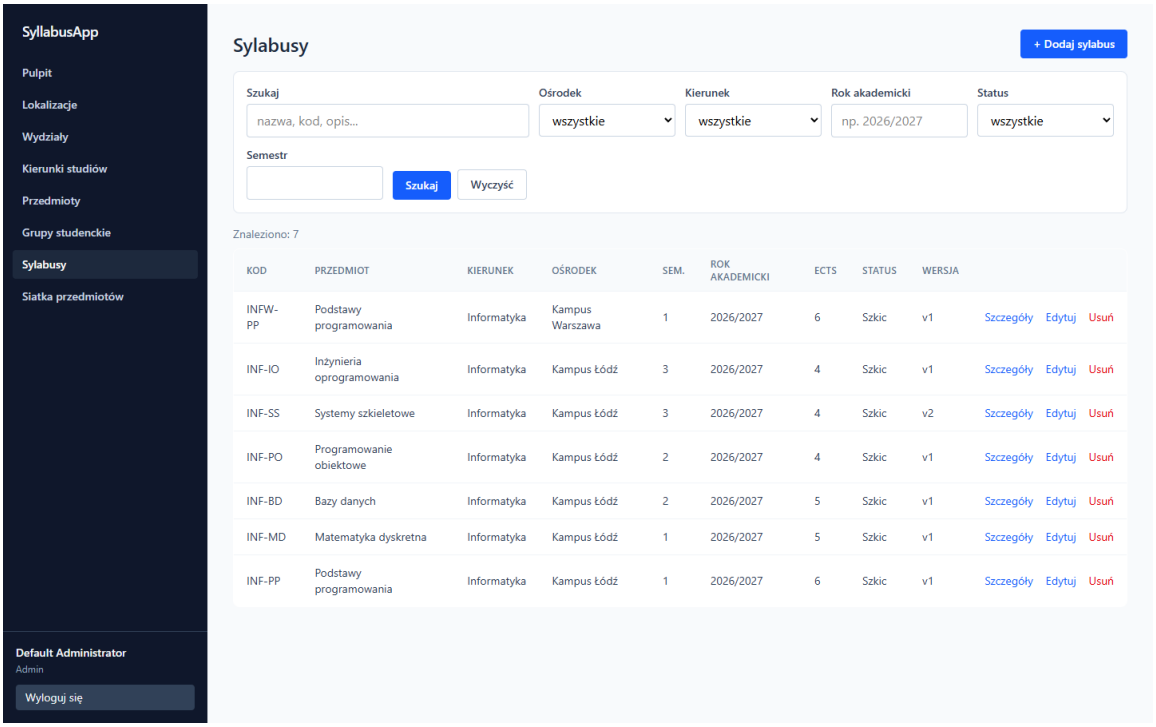
- `Pages/` – strony aplikacji: logowanie i rejestracja, listy i formularze dla ośrodków, wydziałów, kierunków, przedmiotów, grup, sylabusów oraz siatka przedmiotów,
- `Services/ApiClients/` – typowane klienty REST API (po jednym na zasób),
- `Auth/` – `TokenAuthenticationStateProvider` odczytujący role i dane użytkownika z tokenu JWT oraz `AuthorizationMessageHandler` dołączający token do żądań,
- `Layout/` – układ strony z menu bocznym i panelem zalogowanego użytkownika.

Widoczność elementów interfejsu zależy od roli: przyciski dodawania, edycji i usuwania są wyświetlane tylko administratorom i wykładowcom (komponent `AuthorizeView`), a strony wymagające zalogowania mają atrybut `[Authorize]`. Na rysunku 3 przedstawiono ekran logowania.

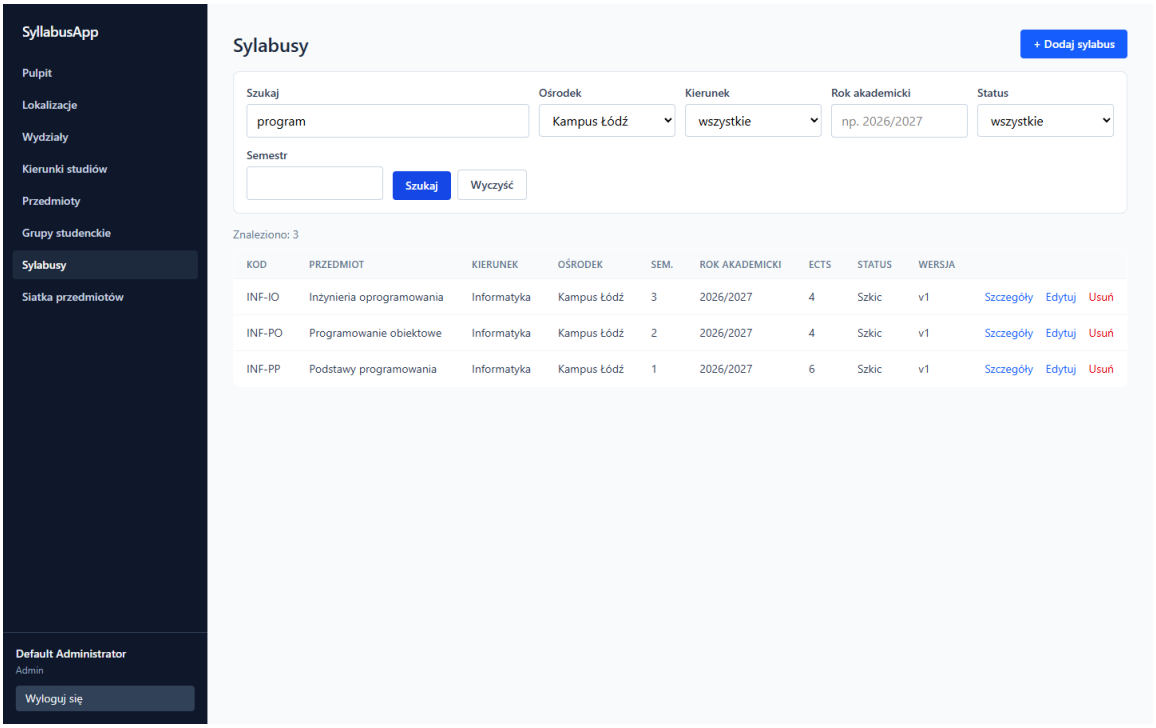


Rysunek 3: Ekran logowania

Lista sylabusów (rys. 4) posiada panel wyszukiwania i filtrowania. Użytkownik może wpisać fragment nazwy lub kodu przedmiotu, opisu albo celów kształcenia, a także zawęzić wyniki według ośrodka, kierunku, roku akademickiego, statusu i semestru. Kryteria są przekazywane do API jako parametry zapytania, a filtrowanie odbywa się w bazie danych (rys. 5).

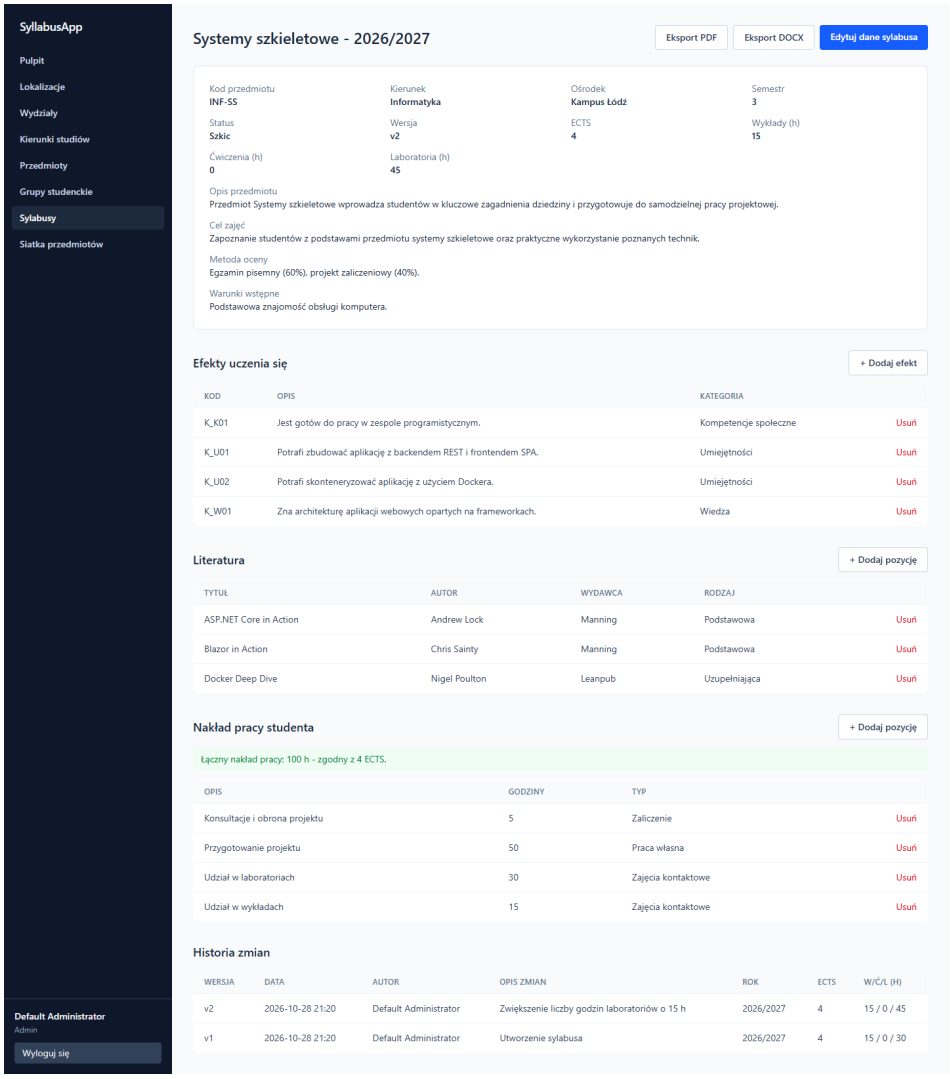


Rysunek 4: Lista sylabusów



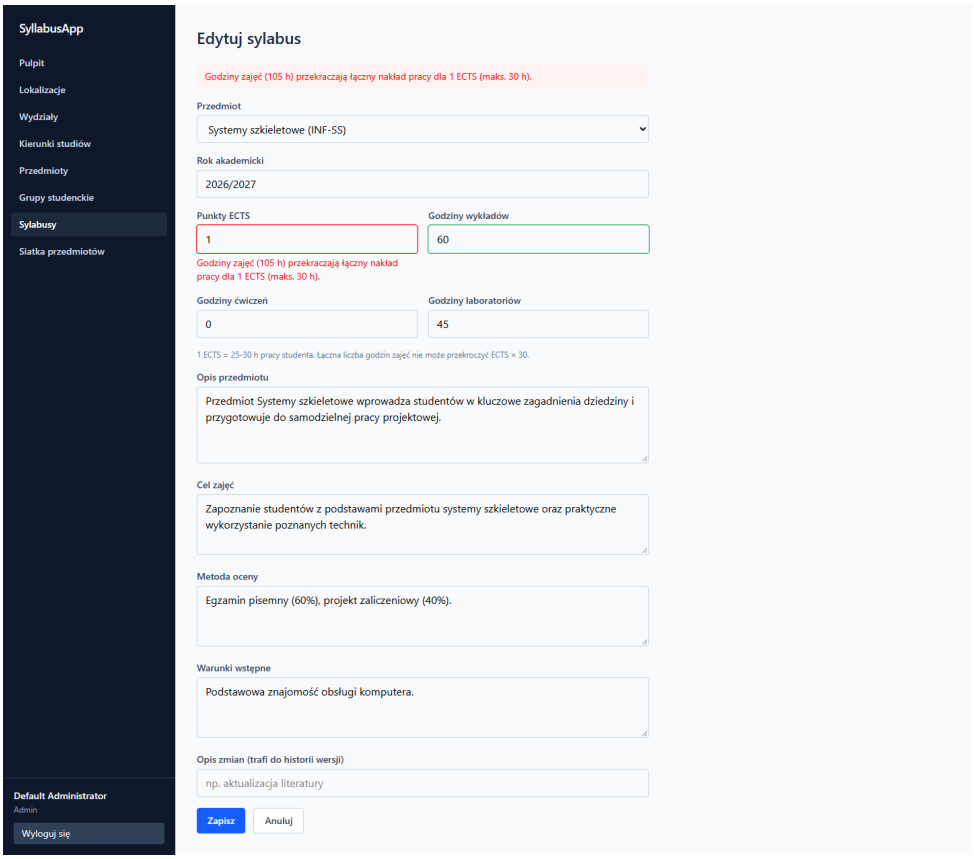
Rysunek 5: Wyszukiwanie i filtrowanie sylabusów (tekst „program”, ośrodek Kampus Łódź)

Strona szczegółów sylabusa (rys. 6) prezentuje dane podstawowe, opisy, efekty uczenia się, literaturę, nakład pracy studenta z automatyczną kontrolą zgodności z ECTS oraz historię zmian. Z tego miejsca można wyeksportować sylabus do PDF lub DOCX – plik jest pobierany przez `HttpClient` (z tokenem JWT), a następnie zapisywany na dysku przez krótką funkcję JavaScript wywoływaną przez `IJSRuntime`.



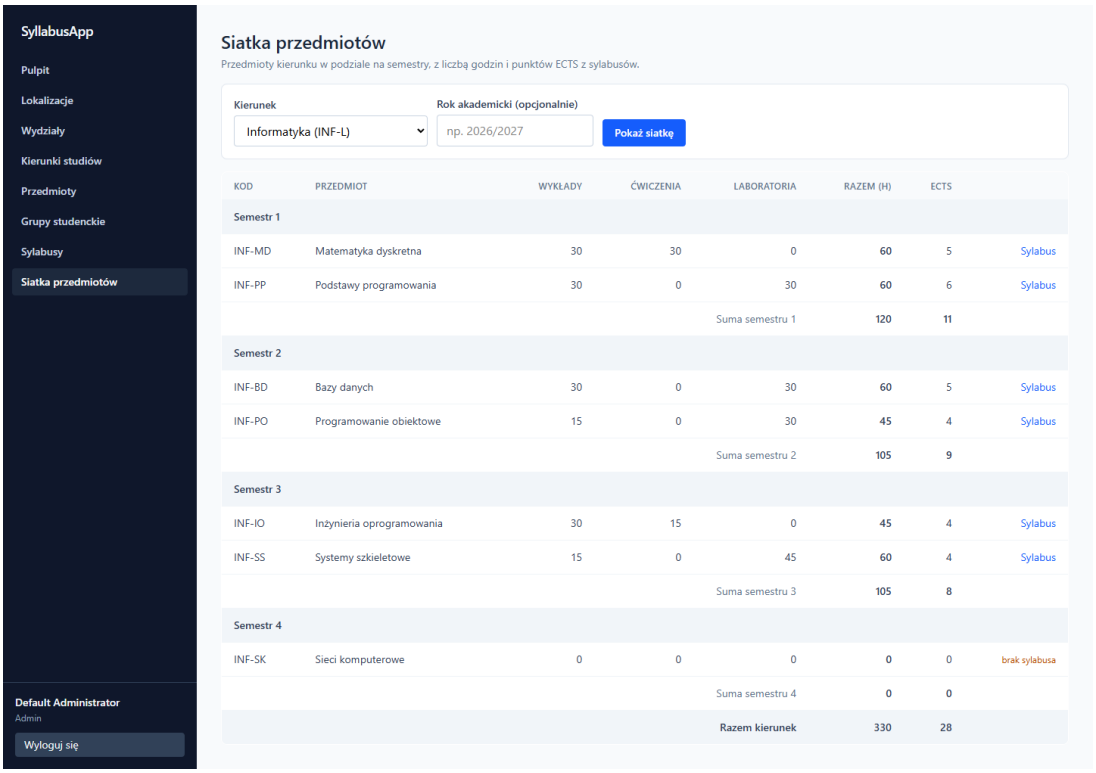
Rysunek 6: Szczegóły sylabusa z nakładem pracy i historią zmian

Formularz sylabusa (rys. 7) korzysta z komponentu `EditForm` i walidatora atrybutów `DataAnnotations`. Ponieważ DTO implementuje interfejs `IValidatableObject`, reguły dotyczące godzin i punktów ECTS są sprawdzane już w przeglądarce, przed wysłaniem żądania. Przy edycji użytkownik może podać opis zmian, który trafia do historii wersji.



Rysunek 7: Walidacja liczby godzin w formularzu sylabusa

Widok siatki przedmiotów (rys. 8) pokazuje przedmioty wybranego kierunku w podziale na semestry, wraz z godzinami wykładów, ćwiczeń i laboratoriów, punktami ECTS oraz sumami dla semestrów i całego kierunku. Przedmioty, które nie mają jeszcze sylabusa, są wyraźnie oznaczone.



Rysunek 8: Siatka przedmiotów kierunku Informatyka

5.2 Backend

Backend jest aplikacją ASP.NET Core Web API [3]. Dla każdego zasobu zastosowano ten sam wzorzec: kontroler przyjmuje i zwraca obiekty DTO, deleguje pracę do serwisu (interfejs w warstwie `Application`, implementacja w `Infrastructure`), a serwis korzysta z `SyllabusDbContext`. Serwisy rejestrowane są w kontenerze wstrzykiwania zależności z cyklem życia *scoped*. Operacje, które mogą zakończyć się błędem biznesowym, zwracają obiekt wyniku (np. `SyllabusResult`) zamiast rzucać wyjątki, a kontroler zamienia go na odpowiedni kod HTTP (200, 201, 400, 404).

5.2.1 Uwierzytelnianie i autoryzacja

Użytkownicy i role są obsługiwane przez ASP.NET Core Identity [21]. Hasło musi mieć co najmniej 8 znaków, w tym cyfrę oraz małą i wielką literę, a po 5 nieudanych próbach konto jest blokowane na 15 minut. Po poprawnym logowaniu `TokenService` generuje token JWT podpisany kluczem HMAC-SHA256, zawierający identyfikator, e-mail, imię, nazwisko i role użytkownika. Nowo zarejestrowany użytkownik otrzymuje rolę *Student*; konto administratora i role są tworzone przy starcie aplikacji przez `DataSeeder`. Operacje modyfikujące dane wymagają roli *Admin* lub *Lecturer* (atrybut `[Authorize(Roles = ...)]`).

5.2.2 Research: logowanie kontem uczelnianym

Wymaganie logowania kontem uczelnianym wymagało analizy dostępnych mechanizmów. Rozważono dwa podejścia:

1. **OpenID Connect z Microsoft Entra ID** [22, 23] – większość polskich uczelni (w tym SAN) udostępnia studentom i pracownikom konta Microsoft 365 w domenie uczelni. Aplikację rejestruje się w Entra ID, a logowanie odbywa się według przepływu *Authorization Code* z PKCE. Po stronie frontendu można użyć biblioteki MSAL dla Blazor WebAssembly, a backend weryfikuje token wydany przez Entra ID (`AddMicrosoftIdentityWebApi`). Ograniczenie dostępu do jednej organizacji realizuje się przez identyfikator dzierżawy (*tenant ID*).

2. **USOS API** [10] – uczelnie korzystające z USOS udostępniają API z autoryzacją OAuth 1.0a. Pozwala ono potwierdzić tożsamość studenta i pobrać jego dane (np. numer albumu, kierunek). Wymaga jednak uzyskania klucza aplikacji od administratora USOS danej uczelni.

Rekomendowanym rozwiązaniem jest OpenID Connect z Entra ID, ponieważ jest oparte na otwartych standardach [24, 22], dobrze wspierane przez platformę .NET i nie wymaga, aby uczelnia korzystała z USOS. W obecnej architekturze wdrożenie polegałoby na dodaniu drugiego schematu uwierzytelniania w API oraz endpointu, który po weryfikacji tokenu Entra ID odnajduje (lub tworzy) lokalnego użytkownika po adresie e-mail i wydaje własny token JWT aplikacji. Dzięki temu reszta systemu (role, autoryzacja, frontend) pozostaje bez zmian. Integracja nie została uruchomiona produkcyjnie, ponieważ wymaga rejestracji aplikacji w dzierżawie uczelni, do której potrzebne są uprawnienia administratora IT uczelni.

5.2.3 Wersjonowanie i historia zmian

Każda operacja zapisu sylabusa tworzy nowy rekord `SyllabusVersion`. Przy utworzeniu powstaje wersja 1 z opisem „Utworzenie sylabusa”, a każda edycja dodaje wersję o numerze o jeden większym od poprzedniej. Wersja przechowuje kopię najważniejszych pól (rok, ECTS, godziny, cele, ocenianie, wymagania), autora, datę, opis zmian podany przez użytkownika oraz pełną migawkę sylabusa w formacie JSON (listing 1). Unikalny indeks na parze (`SyllabusId`, `VersionNumber`) gwarantuje spójność numeracji.

Listing 1: Zapis nowej wersji przy edycji sylabusa (fragment `SyllabusService`)

```
1  syllabus.ApplyUpdate(dto);
2  syllabus.UpdatedAt = DateTime.UtcNow;
3  await _dbContext.SaveChangesAsync();
4
5  // Każda edycja zapisuje nową wersję do historii
6  var lastVersion = await _dbContext.SyllabusVersions
7      .Where(v => v.SyllabusId == id)
```

```
8       .MaxAsync(v => (int?)v.VersionNumber) ?? 0;
9   var description = string.IsNullOrWhiteSpace(dto.ChangeDescription)
10      ? "Aktualizacja sylabusa" : dto.ChangeDescription.Trim();
11  await AddVersionAsync(syllabus, lastVersion + 1, description, userId);
```

5.2.4 Walidacja danych

Walidacja odbywa się na dwóch poziomach. Atrybuty `DataAnnotations` sprawdzają pola wymagane, długość tekstu oraz format roku akademickiego (wyrażenie regularne RRRR/RRRR). Reguły dziedzinowe dotyczące godzin zebrano w klasie `SyllabusRules` (listing 2). Przyjęto, zgodnie z zasadami systemu ECTS [2], że jeden punkt ECTS odpowiada 25–30 godzinom pracy studenta. Na tej podstawie:

- liczba punktów ECTS musi mieścić się w zakresie 1–30,
- liczby godzin nie mogą być ujemne, a sylabus musi mieć co najmniej jedną godzinę zajęć,
- suma godzin zajęć nie może przekroczyć ECTS $\times$ 30,
- suma pozycji nakładu pracy studenta powinna wynosić od ECTS $\times$ 25 do ECTS $\times$ 30 godzin – w przeciwnym razie na stronie sylabusa wyświetlane jest ostrzeżenie.

Listing 2: Reguły walidacji godzin (fragment `SyllabusRules`)

```
1   public static List<string> ValidateHours(int ects, int lectureHours,
2       int exerciseHours, int laboratoryHours)
3   {
4       var errors = new List<string>();
5       if (ects < MinEcts || ects > MaxEcts)
6           errors.Add($"Liczba punktów ECTS musi mieścić się w zakresie {MinEcts}-{
                MaxEcts}.");
7       if (lectureHours < 0 || exerciseHours < 0 || laboratoryHours < 0)
8           errors.Add("Liczba godzin nie może być ujemna.");
9       var contactHours = lectureHours + exerciseHours + laboratoryHours;
10      if (contactHours <= 0)
11          errors.Add("Sylabus musi mieć co najmniej jedną godzinę zajęć.");
12      if (ects >= MinEcts && contactHours > ects * MaxHoursPerEcts)
13          errors.Add($"Godziny zajęć ({contactHours} h) przekraczają ...");
14      return errors;
15  }
```

Klasa bazowa formularza `SyllabusEditDto` implementuje `IValidatableObject` i wywołuje te reguły. Ponieważ projekt `Application` jest współdzielony, ta sama walidacja działa w Blazorze (komunikat pojawia się przy polu formularza) i w API (odpowiedź 400 z opisem błędu).

5.2.5 Wyszukiwanie i siatka przedmiotów

Metoda `GetAllAsync(SyllabusFilterDto)` buduje zapytanie LINQ dynamicznie – każdy niepusty parametr filtra dodaje kolejny warunek `Where`, a całość jest tłumaczona przez

EF Core na jedno zapytanie SQL z odpowiednimi złączeniami (sylabus → przedmiot → kierunek → wydział → ośrodek). Wyszukiwanie tekstowe nie rozróżnia wielkości liter.

Siatka przedmiotów (`GetCurriculumGridAsync`) pobiera wszystkie przedmioty kierunku wraz z sylabusami, grupuje je według semestru i dla każdego przedmiotu wybiera sylabus z podanego roku akademickiego (lub najnowszy, jeśli rok nie został podany). Sumy godzin i ECTS dla semestrów i całego kierunku są wyliczane we właściwościach DTO.

5.2.6 Eksport do PDF i DOCX

Eksport realizuje serwis `SyllabusExportService`. Najpierw ładuje sylabus z wszystkimi powiązanymi danymi (przedmiot, kierunek, wydział, ośrodek, prowadzący, efekty, literatura, nakład pracy), a następnie przygotowuje wspólną treść dokumentu: tabelę danych podstawowych, sekcje opisowe oraz tabele efektów uczenia się, literatury i nakładu pracy. Plik PDF generowany jest biblioteką QuestPDF [17] (licencja Community) z wykorzystaniem płynnego API opisującego układ strony, a plik DOCX – przez Open XML SDK [18], który zapisuje dokument Word bez potrzeby instalowania pakietu Office na serwerze. Oba formaty poprawnie obsługują polskie znaki.

5.3 Baza danych

Jako system bazodanowy wybrano PostgreSQL 16 [5] – dojrzały, otwartoźródłowy system relacyjny, dostępny jako oficjalny obraz Docker. Dostęp do bazy realizuje Entity Framework Core 10 [6] z dostawcą Npgsql [16] w podejściu *code first*: schemat bazy wynika z klas encji i konfiguracji w `SyllabusDbContext`, a zmiany schematu są zapisywane jako migracje. Migracje są stosowane automatycznie przy starcie API (`Database.MigrateAsync()`), dzięki czemu nowa instalacja nie wymaga ręcznego tworzenia tabel.

Tabela 6: Najważniejsze encje modelu danych

Encja	Opis i powiązania
`Location`	Ośrodek akademicki (nazwa, adres, miasto); ma wiele wydziałów
`Faculty`	Wydział w ośrodku; opcjonalnie dziekan (`User`)
`FieldOfStudy`	Kierunek studiów (poziom, liczba semestrów) na wydziale
`Course`	Przedmiot kierunku z numerem semestru
`Syllabus`	Sylabus przedmiotu dla roku akademickiego: ECTS, godziny, opis, cele, ocenianie, wymagania, status, prowadzący
`LearningOutcome`	Efekt uczenia się (kod, opis, kategoria: wiedza, umiejętności, kompetencje społeczne)
`Literature`	Pozycja literatury (podstawowa lub uzupełniająca)
`WorkloadItem`	Pozycja nakładu pracy studenta (zajęcia kontaktowe, praca własna, zaliczenie)
`SyllabusVersion`	Wersja sylabusa: numer, opis zmian, autor, kopia pól i migawka JSON
`SyllabusApproval`	Wpis obiegu akceptacji (przygotowany pod dalszy rozwój)
`StudentGroup`	Grupa studencka na kierunku
`User`, `Role`	Użytkownicy i role (rozszerzenie tabel ASP.NET Core Identity)

Hierarchia `Location` → `Faculty` → `FieldOfStudy` → `Course` → `Syllabus` pozwala przechowywać odrębne sylabusy tego samego przedmiotu dla różnych ośrodków oraz – dzięki polu `AcademicYear` – dla różnych roczników wprowadzenia programu. W konfiguracji modelu zdefiniowano m.in. unikalne indeksy na kodach wydziałów i kierunków, kaskadowe usuwanie wersji razem z sylabusem oraz ograniczenie usuwania użytkownika, który jest prowadzącym lub koordynatorem sylabusa (`DeleteBehavior.Restrict`).

5.4 Docker

Każdy komponent systemu posiada własny obraz kontenera:

- **api** – wieloetapowy `Dockerfile`: w etapie `build` obraz `dotnet/sdk:10.0` wykonuje `dotnet publish`, a wynik kopiowany jest do lekkiego obrazu `dotnet/aspnet:10.0`,
- **web** – w etapie budowania generowany jest arkusz Tailwind CSS i publikowana aplikacja Blazor; pliki statyczne serwuje obraz `nginx:alpine` z regułą `try_files`, która przekierowuje nieznane ścieżki do `index.html` (routing po stronie klienta),
- **db** – oficjalny obraz `postgres:16-alpine` z wolumenem na dane.

Kontenery są opisane w pliku `docker-compose.yml` (listing 3). Konfiguracja API (połączenie z bazą, klucz JWT, konto administratora, dozwolone źródła CORS) jest przekazywana przez zmienne środowiskowe, które w ASP.NET Core nadpisują wartości z `appsettings.json`. API startuje dopiero wtedy, gdy baza zgłosi gotowość (`condition: service_healthy`).

Listing 3: Fragment pliku `docker-compose.yml`

```
  api:
    build:
      context: .
      dockerfile: src/SyllabusApp.Api/Dockerfile
    depends_on:
      db:
        condition: service_healthy
    environment:
      ConnectionStrings__DefaultConnection: "Host=db;Port=5432;..."
      Jwt__Key: "..."
      Cors__Origins__0: "http://localhost:8080"
    ports:
      - "8081:8080"
    healthcheck:
      test: ["CMD-SHELL", "curl -fs http://localhost:8080/health || exit 1"]
      interval: 15s
```

Uruchomienie całego systemu sprowadza się do polecenia `docker compose up -d -build`. Frontend jest wtedy dostępny pod adresem `localhost:8080`, a API pod adresem `localhost:8081`.

5.5 Testy

Do testów wykorzystano framework xUnit [19]. Testy jednostkowe serwisów korzystają z dostawcy EF Core InMemory – każdy test otrzymuje nową, pustą bazę w pamięci (unikalna nazwa z `Guid`), dzięki czemu testy są od siebie niezależne i nie wymagają działającego serwera PostgreSQL. Projekt zawiera 22 testy podzielone na trzy klasy (tab. 7).

Tabela 7: Zakres testów jednostkowych

Klasa testowa	**Liczba**	**Sprawdzane zachowanie**
`SyllabusRulesTests`	10	Zakres ECTS, ujemne i zerowe godziny, przekroczenie limitu godzin, zgodność nakładu pracy z ECTS (przypadki graniczne)
`SyllabusServiceTests`	6	Utworzenie wersji 1, dodanie wersji 2 z zachowaniem starych wartości, błąd dla nieistniejącego przedmiotu, filtrowanie po ośrodku/roku/tekście, siatka przedmiotów, eksport PDF i DOCX
`LocationServiceTests`	6	Operacje CRUD na ośrodkach, sortowanie, obsługa nieistniejących rekordów

Listing 4 przedstawia przykładowy test sprawdzający wersjonowanie. Testy uruchamia się poleceniem `dotnet test`; wszystkie kończą się powodzeniem.

Listing 4: Test wersjonowania sylabusa

```
[Fact]
public async Task UpdateAsync_AddsNewVersion_AndKeepsOldValuesInHistory()
{
```

```
    using var db = CreateDb();
    var service = new SyllabusService(db);
    var dto = NewSyllabus(1);
    var created = (await service.CreateAsync(dto, LecturerId)).Data!;

    var update = ToUpdate(dto, "Więcej godzin laboratoriów");
    update.LaboratoryHours = 45;
    var result = await service.UpdateAsync(created.Id, update, LecturerId);

    Assert.Equal(2, result.Data!.CurrentVersion);
    var versions = (await service.GetVersionsAsync(created.Id))!.ToList();
    Assert.Equal([2, 1], versions.Select(v => v.VersionNumber));
    Assert.Equal(45, versions[0].LaboratoryHours);
    Assert.Equal(30, versions[1].LaboratoryHours);
}
```

5.6 Monitoring

Stan systemu jest monitorowany za pomocą mechanizmu *health checks* wbudowanego w ASP.NET Core [20]. W API zarejestrowano sprawdzenie `AddDbContextCheck` dla kontekstu `SyllabusDbContext`, które przy każdym wywołaniu endpointu `/health` weryfikuje, czy aplikacja może połączyć się z bazą danych. Endpoint zwraca `Healthy` (kod 200) lub `Unhealthy` (kod 503).

Listing 5: Konfiguracja health checks w `Program.cs`

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<SyllabusDbContext>("database");
// ...
app.MapHealthChecks("/health");
```

Endpoint jest wykorzystywany przez Docker: healthcheck kontenera `api` co 15 sekund odpytuje `/health`, a kontener bazy jest sprawdzany poleceniem `pg_isready`. Aktualny stan wszystkich usług (`healthy` / `unhealthy`) można zobaczyć poleceniem `docker compose ps`, a w razie awarii Docker automatycznie restartuje kontenery (`restart: unless-stopped`). Dodatkowo aplikacja zapisuje logi (m.in. wykonywane zapytania SQL, błędy, tworzenie kont), dostępne poleceniem `docker compose logs api`.

6 Podsumowanie i wnioski

W ramach projektu zbudowano kompletną aplikację webową do zarządzania sylabusami uczelni wyższej. Zrealizowano wszystkie elementy wymagane w projekcie: frontend (Blazor WebAssembly), backend (ASP.NET Core Web API), bazę danych (PostgreSQL z EF Core), konteneryzację (Docker Compose), testy jednostkowe (xUnit) oraz monitoring stanu aplikacji (health checks). Spełniono również wymagania funkcjonalne: formularze sylabusa, wersjonowanie z historią zmian, obsługę wielu ośrodków i roczników, siatkę przedmiotów, walidację godzin, wyszukiwanie i filtrowanie oraz eksport do PDF i DOCX. Dla logowania kontem uczelnianym przeprowadzono research i zaproponowano sposób integracji z Microsoft Entra ID.

6.1 Napotkane wyzwania

- **Współdzielenie walidacji** – aby nie powielać reguł, formularze tworzenia i edycji sylabusa oparto na wspólnej klasie bazowej z `IValidatableObject`, umieszczonej w projekcie używanym zarówno przez API, jak i Blazor.
- **Enumeracje w JSON** – domyślnie .NET serializuje enumeracje jako liczby; konieczne było spójne skonfigurowanie `JsonStringEnumConverter` po obu stronach.
- **Pobieranie plików z autoryzacją** – zwykły link nie przesyła tokenu JWT, dlatego eksport pobierany jest przez `HttpClient`, a zapis pliku realizuje krótka funkcja JavaScript.
- **Konteneryzacja frontendu** – arkusz Tailwind CSS nie jest przechowywany w repozytorium, dlatego jest generowany w trakcie budowania obrazu, a adres API wstrzykiwany jest do `appsettings.json` jako argument budowania.
- **Pierwsze uruchomienie** – pusta baza w kontenerze wymagała automatycznego stosowania migracji przy starcie API.

6.2 Możliwości dalszego rozwoju

- wdrożenie logowania kontem uczelnianym (OpenID Connect, Microsoft Entra ID) zgodnie z opisanym researchem,
- obieg akceptacji sylabusa (wykładowca → dziekan) z wykorzystaniem przygotowanej encji `SyllabusApproval` i statusów,
- porównywanie dwóch wersji sylabusa i przywracanie wybranej wersji,
- publiczny katalog opublikowanych sylabusów dostępny bez logowania,
- rozbudowa monitoringu o metryki (Prometheus, Grafana) i scentralizowane logi,

- testy integracyjne API oraz testy interfejsu użytkownika,
- powiązanie efektów uczenia się przedmiotu z efektami kierunkowymi i Polską Ramą Kwalifikacji.

Projekt pokazał, że współczesna platforma .NET pozwala zbudować pełną aplikację webową – od bazy danych po interfejs użytkownika – w jednym języku programowania, a konteneryzacja znacznie upraszcza jej uruchomienie i wdrożenie.

Spis tabel

1 Wymagania funkcjonalne i sposób ich realizacji 4
2 Porównanie istniejących rozwiązań z SyllabusApp 7
3 Projekty (warstwy) rozwiązania 9
4 Najważniejsze endpointy REST API 10
5 Stos technologiczny 11
6 Najważniejsze encje modelu danych 20
7 Zakres testów jednostkowych 21

Spis rysunków

1 Architektura systemu SyllabusApp 8
2 Interaktywna dokumentacja API (Scalar) 10
3 Ekran logowania 12
4 Lista sylabusów 13
5 Wyszukiwanie i filtrowanie sylabusów (tekst „program”, ośrodek Kampus Łódź) 13
6 Szczegóły sylabusa z nakładem pracy i historią zmian 14
7 Walidacja liczby godzin w formularzu sylabusa 15
8 Siatka przedmiotów kierunku Informatyka 16

Spis listingów

1 Zapis nowej wersji przy edycji sylabusa (fragment `SyllabusService`) 17
2 Reguły walidacji godzin (fragment `SyllabusRules`) 18
3 Fragment pliku `docker-compose.yml` 21
4 Test wersjonowania sylabusa 21
5 Konfiguracja health checks w `Program.cs` 22

Bibliografia

[1] Sejm RP. *Ustawa z dnia 20 lipca 2018 r. – Prawo o szkolnictwie wyższym i nauce (Dz.U. 2018 poz. 1668)*. 2018. URL: https://isap.sejm.gov.pl/isap.nsf/DocDetails.xsp?id=WDU20180001668 (dostęp 28.10.2026).

[2] European Commission. *European Credit Transfer and Accumulation System (ECTS)*. URL: https://education.ec.europa.eu/education-levels/higher-education/inclusive-and-connected-higher-education/european-credit-transfer-and-accumulation-system (dostęp 28.10.2026).

[3] Microsoft. *ASP.NET Core documentation*. URL: https://learn.microsoft.com/aspnet/core/ (dostęp 28.10.2026).

[4] Microsoft. *ASP.NET Core Blazor*. URL: https://learn.microsoft.com/aspnet/core/blazor/ (dostęp 28.10.2026).

[5] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. URL: https://www.postgresql.org/docs/16/ (dostęp 28.10.2026).

[6] Microsoft. *Entity Framework Core*. URL: https://learn.microsoft.com/ef/core/ (dostęp 28.10.2026).

[7] Docker Inc. *Docker Docs – Docker Compose*. URL: https://docs.docker.com/compose/ (dostęp 28.10.2026).

[8] R. C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[9] Międzyuniwersyteckie Centrum Informatyzacji. *Uniwersytecki System Obsługi Studiów USOS*. URL: https://www.usos.edu.pl/ (dostęp 28.10.2026).

[10] Międzyuniwersyteckie Centrum Informatyzacji. *USOS API – dokumentacja*. URL: https://apps.usos.edu.pl/developers/api/ (dostęp 28.10.2026).

[11] Akademus. *Akademus – system wspomagający zarządzanie uczelnią*. URL: https://www.akademus.pl/ (dostęp 28.10.2026).

[12] Asseco Data Systems. *Uczelnia.XP – system zarządzania uczelnią*. URL: https://www.uczelnia.xp.pl/ (dostęp 28.10.2026).

[13] M. Jones, J. Bradley i N. Sakimura. *RFC 7519: JSON Web Token (JWT)*. 2015. URL: https://www.rfc-editor.org/rfc/rfc7519 (dostęp 28.10.2026).

[14] Scalar. *Scalar API Reference for .NET*. URL: https://github.com/scalar/scalar (dostęp 28.10.2026).

[15] Tailwind Labs. *Tailwind CSS Documentation*. URL: https://tailwindcss.com/docs (dostęp 28.10.2026).

[16] Npgsql Development Team. *Npgsql Entity Framework Core Provider*. URL: https://www.npgsql.org/efcore/ (dostęp 28.10.2026).

[17] QuestPDF. *QuestPDF – Modern PDF library for C# developers*. URL: https://www.questpdf.com/ (dostęp 28.10.2026).

[18] Microsoft. *Open XML SDK*. URL: https://learn.microsoft.com/office/open-xml/open-xml-sdk (dostęp 28.10.2026).

[19] .NET Foundation. *xUnit.net – Getting Started*. URL: https://xunit.net/ (dostęp 28.10.2026).

[20] Microsoft. *Health checks in ASP.NET Core*. URL: https://learn.microsoft.com/aspnet/core/host-and-deploy/health-checks (dostęp 28.10.2026).

[21] Microsoft. *Introduction to Identity on ASP.NET Core*. URL: https://learn.microsoft.com/aspnet/core/security/authentication/identity (dostęp 28.10.2026).

[22] OpenID Foundation. *OpenID Connect Core 1.0*. URL: https://openid.net/specs/openid-connect-core-1_0.html (dostęp 28.10.2026).

[23] Microsoft. *Microsoft identity platform and OpenID Connect protocol*. URL: https://learn.microsoft.com/entra/identity-platform/v2-protocols-oidc (dostęp 28.10.2026).

[24] D. Hardt. *RFC 6749: The OAuth 2.0 Authorization Framework*. 2012. URL: https://www.rfc-editor.org/rfc/rfc6749 (dostęp 28.10.2026).